Презентация доклада

Методы защиты от атаки типа переполнение буфера.

Бессонов Андрей Максимович

2026-03-27

Содержание I

1. Информация
2. Вводная часть
3. Основная часть
4. Заключение

Раздел 1

Информация

Докладчик

- Бессонов Андрей Максимович

Докладчик

- Бессонов Андрей Максимович
- Студент 1-го курса

Докладчик

- Бессонов Андрей Максимович
- Студент 1-го курса
- Группа НКАбд-01-25

Докладчик

- Бессонов Андрей Максимович
- Студент 1-го курса
- Группа НКАбд-01-25
- Российский университет дружбы народов им. П. Лумумбы

Раздел 2

Вводная часть

Актуальность

- Переполнение буфера остаётся одной из самых опасных уязвимостей в UNIX-системах на протяжении десятилетий.

Актуальность

- Переполнение буфера остаётся одной из самых опасных уязвимостей в UNIX-системах на протяжении десятилетий.
- Несмотря на развитие технологий защиты, ошибки работы с памятью в C/C++ продолжают встречаться в коде ядра и приложений.

Актуальность

- Переполнение буфера остаётся одной из самых опасных уязвимостей в UNIX-системах на протяжении десятилетий.
- Несмотря на развитие технологий защиты, ошибки работы с памятью в C/C++ продолжают встречаться в коде ядра и приложений.
- Атаки этого типа позволяют злоумышленнику выполнить произвольный код и получить повышенные привилегии.

Актуальность

- Переполнение буфера остаётся одной из самых опасных уязвимостей в UNIX-системах на протяжении десятилетий.
- Несмотря на развитие технологий защиты, ошибки работы с памятью в C/C++ продолжают встречаться в коде ядра и приложений.
- Атаки этого типа позволяют злоумышленнику выполнить произвольный код и получить повышенные привилегии.
- Для обеспечения безопасности необходимы комплексные методы защиты: компиляторные, аппаратные и организационные.

Объект и предмет исследования

- **Объект:** Механизмы защиты операционных систем семейства UNIX от атак типа переполнение буфера.

Объект и предмет исследования

- **Объект:** Механизмы защиты операционных систем семейства UNIX от атак типа переполнение буфера.
- **Предмет:** Современные методы защиты на этапе компиляции, уровне ОС и процессора, а также подходы к безопасному программированию.

Цели и задачи

- **Цель:** Рассмотреть современные методы защиты от атак типа переполнение буфера, применяемые в среде UNIX, и оценить их эффективность.

Цели и задачи

- **Цель:** Рассмотреть современные методы защиты от атак типа переполнение буфера, применяемые в среде UNIX, и оценить их эффективность.
- **Задачи:**

Цели и задачи

- **Цель:** Рассмотреть современные методы защиты от атак типа переполнение буфера, применяемые в среде UNIX, и оценить их эффективность.
- **Задачи:**
  1. Изучить причины возникновения уязвимости и принцип классической атаки (stack overflow).

Цели и задачи

- **Цель:** Рассмотреть современные методы защиты от атак типа переполнение буфера, применяемые в среде UNIX, и оценить их эффективность.
- **Задачи:**
  1. Изучить причины возникновения уязвимости и принцип классической атаки (stack overflow).
  2. Проанализировать компиляторные методы защиты (StackGuard, ASLR).

Цели и задачи

- **Цель:** Рассмотреть современные методы защиты от атак типа переполнение буфера, применяемые в среде UNIX, и оценить их эффективность.
- **Задачи:**
  1. Изучить причины возникновения уязвимости и принцип классической атаки (stack overflow).
  2. Проанализировать компиляторные методы защиты (StackGuard, ASLR).
  3. Рассмотреть аппаратно-системные средства (NX-бит, изоляция памяти).

Цели и задачи

- **Цель:** Рассмотреть современные методы защиты от атак типа переполнение буфера, применяемые в среде UNIX, и оценить их эффективность.
- **Задачи:**
  1. Изучить причины возникновения уязвимости и принцип классической атаки (stack overflow).
  2. Проанализировать компиляторные методы защиты (StackGuard, ASLR).
  3. Рассмотреть аппаратно-системные средства (NX-бит, изоляция памяти).
  4. Охарактеризовать меры безопасного программирования и анализа кода.

Цели и задачи

- **Цель:** Рассмотреть современные методы защиты от атак типа переполнение буфера, применяемые в среде UNIX, и оценить их эффективность.
- **Задачи:**
  1. Изучить причины возникновения уязвимости и принцип классической атаки (stack overflow).
  2. Проанализировать компиляторные методы защиты (StackGuard, ASLR).
  3. Рассмотреть аппаратно-системные средства (NX-бит, изоляция памяти).
  4. Охарактеризовать меры безопасного программирования и анализа кода.
  5. Сделать выводы о необходимости комплексного подхода.

Материалы и методы

- **Оборудование:** Виртуальная или физическая машина с ОС Linux (Ubuntu/Fedora).

Материалы и методы

- **Оборудование:** Виртуальная или физическая машина с ОС Linux (Ubuntu/Fedora).
- **Программное обеспечение:** Компилятор GCC, ядро Linux, инструменты статического анализа (Splint, Coverity), динамические анализаторы (Valgrind, AddressSanitizer).

Материалы и методы

- **Оборудование:** Виртуальная или физическая машина с ОС Linux (Ubuntu/Fedora).
- **Программное обеспечение:** Компилятор GCC, ядро Linux, инструменты статического анализа (Splint, Coverity), динамические анализаторы (Valgrind, AddressSanitizer).
- **Методы:** Анализ технической документации, экспериментальная проверка работы защитных механизмов, изучение публикаций по безопасности.

Раздел 3

Основная часть

Переполнение буфера как угроза безопасности UNIX

Причины возникновения уязвимости

- Языки C/C++ предоставляют прямой доступ к памяти и не контролируют границы массивов.

```
НОРМАЛЬНЫЕ ВХОДНЫЕ ДАННЫЕ: AAAABBBBCCCC
ВХОДНЫЕ ДАННЫЕ ЗЛОУМЫШЛЕННИКА: AAAABBBBCCCCDDDD

                НОРМА                 АТАКА
--> +-----------+          +-----------+ <--
|   |   AAAA    |          |   AAAA    |   |
```

Переполнение буфера как угроза безопасности UNIX

Причины возникновения уязвимости

- Языки C/C++ предоставляют прямой доступ к памяти и не контролируют границы массивов.
- Функции `strcpy()`, `strcat()`, `sprintf()`, `gets()` не проверяют размер буфера-приёмника.

```
НОРМАЛЬНЫЕ ВХОДНЫЕ ДАННЫЕ: AAAABBBBCCCC
ВХОДНЫЕ ДАННЫЕ ЗЛОУМЫШЛЕННИКА: AAAABBBBCCCCDDDD

                НОРМА                 АТАКА
--> +-----------+          +-----------+ <--
|   |   AAAA    |          |   AAAA    |   |
```

Переполнение буфера как угроза безопасности UNIX

Причины возникновения уязвимости

- Языки C/C++ предоставляют прямой доступ к памяти и не контролируют границы массивов.
- Функции `strcpy()`, `strcat()`, `sprintf()`, `gets()` не проверяют размер буфера-приёмника.
- Запись данных за пределы буфера приводит к повреждению соседних областей памяти.

```
НОРМАЛЬНЫЕ ВХОДНЫЕ ДАННЫЕ: AAAABBBBCCCC
ВХОДНЫЕ ДАННЫЕ ЗЛОУМЫШЛЕННИКА: AAAABBBBCCCCDDDD

                НОРМА                 АТАКА
        --> +-----------+         +-----------+ <--
        |   |   AAAA    |         |   AAAA    |   |
```

Переполнение буфера как угроза безопасности UNIX

Причины возникновения уязвимости

- Языки C/C++ предоставляют прямой доступ к памяти и не контролируют границы массивов.
- Функции `strcpy()`, `strcat()`, `sprintf()`, `gets()` не проверяют размер буфера-приёмника.
- Запись данных за пределы буфера приводит к повреждению соседних областей памяти.
- Особую опасность представляют SUID-программы, позволяющие локальному пользователю повысить свои привилегии.

```
НОРМАЛЬНЫЕ ВХОДНЫЕ ДАННЫЕ: AAAABBBBCCCC
ВХОДНЫЕ ДАННЫЕ ЗЛОУМЫШЛЕННИКА: AAAABBBBCCCCDDDD

                НОРМА                 АТАКА
--> +-----------+          +-----------+ <--
|   |   AAAA    |          |   AAAA    |   |
```

Принцип классической атаки (stack overflow)

1. В локальный буфер записывается блок данных большего размера, чем выделено.

После завершения функции управление передаётся не легитимному коду, а коду злоумышленника.

Принцип классической атаки (stack overflow)

1. В локальный буфер записывается блок данных большего размера, чем выделено.
2. В этот блок помещается вредоносный код (шелл-код).

После завершения функции управление передаётся не легитимному коду, а коду злоумышленника.

Принцип классической атаки (stack overflow)

1. В локальный буфер записывается блок данных большего размера, чем выделено.
2. В этот блок помещается вредоносный код (шелл-код).
3. Адрес возврата подменяется указателем на начало шелл-кода.

После завершения функции управление передаётся не легитимному коду, а коду злоумышленника.

Методы защиты: компиляторные техники

StackGuard (канарейки)

- Расширение компилятора GCC.

ASLR (Address Space Layout Randomization)

Методы защиты: компиляторные техники

StackGuard (канарейки)

- Расширение компилятора GCC.
- Между локальными переменными и адресом возврата размещается специальное значение – «канарейка».

ASLR (Address Space Layout Randomization)

Методы защиты: компиляторные техники

StackGuard (канарейки)

- Расширение компилятора GCC.
- Между локальными переменными и адресом возврата размещается специальное значение – «канарейка».
- Перед выходом из функции проверяется целостность канарейки; при изменении программа завершается.

ASLR (Address Space Layout Randomization)

Методы защиты: компиляторные техники

StackGuard (канарейки)

- Расширение компилятора GCC.
- Между локальными переменными и адресом возврата размещается специальное значение – «канарейка».
- Перед выходом из функции проверяется целостность канарейки; при изменении программа завершается.

ASLR (Address Space Layout Randomization)

- Рандомизация размещения сегментов стека, кучи, библиотек и исполняемого файла при каждом запуске.

Методы защиты: компиляторные техники

StackGuard (канарейки)

- Расширение компилятора GCC.
- Между локальными переменными и адресом возврата размещается специальное значение – «канарейка».
- Перед выходом из функции проверяется целостность канарейки; при изменении программа завершается.

ASLR (Address Space Layout Randomization)

- Рандомизация размещения сегментов стека, кучи, библиотек и исполняемого файла при каждом запуске.
- Затрудняет предсказание адресов для перехода на шелл-код.

Методы защиты: компиляторные техники

StackGuard (канарейки)

- Расширение компилятора GCC.
- Между локальными переменными и адресом возврата размещается специальное значение – «канарейка».
- Перед выходом из функции проверяется целостность канарейки; при изменении программа завершается.

ASLR (Address Space Layout Randomization)

- Рандомизация размещения сегментов стека, кучи, библиотек и исполняемого файла при каждом запуске.
- Затрудняет предсказание адресов для перехода на шелл-код.
- Наиболее эффективна в 64-разрядных системах.

Сравнение методов компиляторной защиты

Метод	Суть	Эффективность
StackGuard	Контрольная сумма (канарейка) в стеке	Высокая против stack overflow
ASLR	Рандомизация адресного пространства	Высокая при 64-разрядной архитектуре

Аппаратно-системные средства

NX-бит (No-eXecute) / XD-бит

- Аппаратный запрет на исполнение кода из областей памяти, помеченных как данные.

Аппаратно-системные средства

NX-бит (No-eXecute) / XD-бит

- Аппаратный запрет на исполнение кода из областей памяти, помеченных как данные.
- ОС может пометить страницы стека и кучи как неисполняемые.

Аппаратно-системные средства

NX-бит (No-eXecute) / XD-бит

- Аппаратный запрет на исполнение кода из областей памяти, помеченных как данные.
- ОС может пометить страницы стека и кучи как неисполняемые.
- При попытке выполнить код из такой области процессор генерирует исключение.

Аппаратно-системные средства

NX-бит (No-eXecute) / XD-бит

- Аппаратный запрет на исполнение кода из областей памяти, помеченных как данные.
- ОС может пометить страницы стека и кучи как неисполняемые.
- При попытке выполнить код из такой области процессор генерирует исключение.
- В сочетании с ASLR делает большинство классических атак неработоспособными.

Аппаратно-системные средства

NX-бит (No-eXecute) / XD-бит

- Аппаратный запрет на исполнение кода из областей памяти, помеченных как данные.
- ОС может пометить страницы стека и кучи как неисполняемые.
- При попытке выполнить код из такой области процессор генерирует исключение.
- В сочетании с ASLR делает большинство классических атак неработоспособными.
- Не защищает от сложных методов (например, return-oriented programming).

Безопасное программирование и анализ кода

- Использование безопасных версий функций:

Безопасное программирование и анализ кода

- Использование безопасных версий функций:
  - `strncpy()`, `strncat()` вместо `strcpy()`, `strcat()`

Безопасное программирование и анализ кода

- Использование безопасных версий функций:
  - `strncpy()`, `strncat()` вместо `strcpy()`, `strcat()`
  - функции с ограничением длины (snprintf, strlcpy)

Безопасное программирование и анализ кода

- Использование безопасных версий функций:
  - `strncpy()`, `strncat()` вместо `strcpy()`, `strcat()`
  - функции с ограничением длины (snprintf, strlcpy)
- Применение современных языков с автоматическим управлением памятью (Rust, Go).

Безопасное программирование и анализ кода

- Использование безопасных версий функций:
  - `strncpy()`, `strncat()` вместо `strcpy()`, `strcat()`
  - функции с ограничением длины (snprintf, strlcpy)
- Применение современных языков с автоматическим управлением памятью (Rust, Go).
- Регулярное использование статических анализаторов (Splint, Coverity) и динамических анализаторов (Valgrind, AddressSanitizer).

Безопасное программирование и анализ кода

- Использование безопасных версий функций:
  - `strncpy()`, `strncat()` вместо `strcpy()`, `strcat()`
  - функции с ограничением длины (snprintf, strlcpy)
- Применение современных языков с автоматическим управлением памятью (Rust, Go).
- Регулярное использование статических анализаторов (Splint, Coverity) и динамических анализаторов (Valgrind, AddressSanitizer).
- Позволяет устранить уязвимости на этапе разработки.

Раздел 4

Заключение

Результаты работы

В ходе работы были рассмотрены и проанализированы:

1. **Причины возникновения уязвимости** – особенности C/C++, отсутствие контроля границ массивов.

Результаты работы

В ходе работы были рассмотрены и проанализированы:

1. **Причины возникновения уязвимости** – особенности C/C++, отсутствие контроля границ массивов.
2. **Принцип классической атаки** – перезапись адреса возврата для выполнения шелл-кода.

Результаты работы

В ходе работы были рассмотрены и проанализированы:

1. **Причины возникновения уязвимости** – особенности C/C++, отсутствие контроля границ массивов.
2. **Принцип классической атаки** – перезапись адреса возврата для выполнения шелл-кода.
3. **Методы защиты на этапе компиляции** – StackGuard, ASLR, их эффективность.

Результаты работы

В ходе работы были рассмотрены и проанализированы:

1. **Причины возникновения уязвимости** – особенности C/C++, отсутствие контроля границ массивов.
2. **Принцип классической атаки** – перезапись адреса возврата для выполнения шелл-кода.
3. **Методы защиты на этапе компиляции** – StackGuard, ASLR, их эффективность.
4. **Аппаратно-системные средства** – NX-бит, его роль в предотвращении выполнения кода в данных.

Результаты работы

В ходе работы были рассмотрены и проанализированы:

1. **Причины возникновения уязвимости** – особенности C/C++, отсутствие контроля границ массивов.
2. **Принцип классической атаки** – перезапись адреса возврата для выполнения шелл-кода.
3. **Методы защиты на этапе компиляции** – StackGuard, ASLR, их эффективность.
4. **Аппаратно-системные средства** – NX-бит, его роль в предотвращении выполнения кода в данных.
5. **Организационные меры** – безопасное программирование, использование анализаторов, современных языков.

Вывод

- Проблема переполнения буфера не утратила актуальности, но современные методы защиты позволяют минимизировать риски.

Вывод

- Проблема переполнения буфера не утратила актуальности, но современные методы защиты позволяют минимизировать риски.
- Ключевые элементы защиты:

Вывод

- Проблема переполнения буфера не утратила актуальности, но современные методы защиты позволяют минимизировать риски.
- Ключевые элементы защиты:
  - **Компиляторные техники** (StackGuard, ASLR) – усложняют эксплуатацию.

Вывод

- Проблема переполнения буфера не утратила актуальности, но современные методы защиты позволяют минимизировать риски.
- Ключевые элементы защиты:
  - **Компиляторные техники** (StackGuard, ASLR) – усложняют эксплуатацию.
  - **Аппаратная поддержка** (NX-бит) – предотвращает выполнение кода в данных.

Вывод

- Проблема переполнения буфера не утратила актуальности, но современные методы защиты позволяют минимизировать риски.
- Ключевые элементы защиты:
  - **Компиляторные техники** (StackGuard, ASLR) – усложняют эксплуатацию.
  - **Аппаратная поддержка** (NX-бит) – предотвращает выполнение кода в данных.
  - **Организационные меры** – устраняют причины появления багов.

Вывод

- Проблема переполнения буфера не утратила актуальности, но современные методы защиты позволяют минимизировать риски.
- Ключевые элементы защиты:
  - **Компиляторные техники** (StackGuard, ASLR) – усложняют эксплуатацию.
  - **Аппаратная поддержка** (NX-бит) – предотвращает выполнение кода в данных.
  - **Организационные меры** – устраняют причины появления багов.
- Только комплексное применение всех трёх уровней обеспечивает надёжную защиту UNIX-систем от атак, основанных на переполнении буфера.